

Intracranial Hemorrhage Detection Using Vision Transformer and Transformer

Group Members: Amirhossein Rasoulia - Bahar Jahani

dept. of computer science and software engineering

Concordia University

Montreal, Canada

Abstract—Intracranial hemorrhage (ICH) is a serious health problem requiring rapid and often intensive medical treatment. The diagnostic is routinely done using head CT images. The use of artificial intelligence technology would assist the medical community in identifying the presence of hemorrhage and effective decision-making. In this work, to process 3D CT scans, we designed a model consisting of a vision transformer to extract features from slices, and a transformer to extract dependencies between slices. We only use about 4% of actual dataset, and we are getting comparable results which is considerable. To increase the model generalizability, random gaussian noise addition, transfer learning, and weight decay techniques have been used. Besides, to have a better control on training, early stopping and learning rate scheduling was applied. We showed that in our transformer based model, the Sharpness Aware Minimization (SAM) optimizer overcomes the Adam optimizer in performance by giving smoother train-validation curves, avoiding overfitting, and resulting a better final prediction. Also, in this binary classification, using Binary Cross Entropy (BCE) loss function gives us better results compared to Cross Entropy (CE) because the final layer seems to be able to more efficiently map the features to one neuron compared to two neurons. Finally, The vision transformer alone could achieve 85.06% accuracy while using a transformer as a sequence model did not help to achieve a better result.

Index Terms—Hemorrhage, Transformer, Vision Transformer, Binary Cross Entropy, Sharpness-Aware Minimization

I. INTRODUCTION

According to the 'Centers for Disease Control and Prevention' statistics, every year about 795,000 people in United States have a stroke. Around 10% of strokes are Intracranial hemorrhage (ICH) and approximately 50% mortality rate in two years is reported for it. Therefore, there is high risk for ICH to turn into a secondary brain injury or even death if it is not treated in a time sensitive procedure. Using Deep Learning methods can help faster diagnosis of ICH and better treatment as a result.

In the current clinical practices, Computerized Tomography (CT) scans are used for patient's evaluation because of their better availability and acquisition time compared to Magnetic Resonance Imaging (MRI). CT scans are displayed using a windowing method. This method transforms the Hounsfield Units (HU) numbers into grayscale values ($[0, 255]$) according to the window level and width parameters. By selecting different window parameters, different features of the brain tissues are displayed in the grayscale image (e.g., brain window, stroke window, and bone window) [1].

The main difficulty of dealing with CT scans is their 3D representation as training 3D models to extract features directly from voxels is computationally expensive, and it requires a very large dataset. A naive approach is to consider 2D slices independently and classify them whether they include hemorrhage or not. However, we can also use 2D models to extract 2D features from slices, and treat features of different slices as sequences to be trained by a sequential model such as transformer.

Vaswani et al. [2] have defined Transformers with promising results in sequence translation and Dosovitskiy et al. [3] have shown that Vision Transformers (ViT) outperform CNNs in capturing long-range dependencies of images when there is sufficient data. In this paper, first we fine-tune a pre-trained ViT model to classify ICH in 2D slices, then we monitor the effect of using transformer for capturing information in 3D data from feature maps of ViT.

In section II, we summarize three main papers that we have used in our work. In sections III, IV and V, our model architecture, data preparation and training parameters are proposed respectively. Sections VI and VII explain how the model is trained and what the results are, and final conclusion is brought in section VIII.

II. LITERATURE REVIEW

A. Transformer and Attention

"Attention is all you need" [2] is one of the most significant papers in the field of Natural Language Processing which is authored by professionals from the Google research team Vaswani et al., and accepted in 2017 NIPS. This paper was a great improvement in language translation speed and accuracy by introducing the first transduction model architecture relying entirely on an attention mechanism, called Transformer. The popular approaches in Sequence Modeling tasks such as Recurrent Neural Networks (RNN), Long Short-Term Memory Networks (LSTM), and Gated Recurrent Units (GRU) are great at capturing the sequential nature of languages and exploiting words locality. However, this sequentiality is a barrier to parallelization and also has a great limitation in establishing long-range dependencies.

This paper removes recurrence and introduces the usage of Attention Mechanisms to deal with this limitation and to draw global dependencies between input and output.

Transformer is composed of an encoder and a decoder, the same as the general structure for most neural sequence transduction models, implemented by an architecture using stacked self-attention, point-wise, and fully connected layers for both the encoder and decoder. Each encoder and decoder have $N=6$ layers and each layer has some sub-layers which have a residual connection, followed by layer norm. Fig. 1 shows details of the architecture.

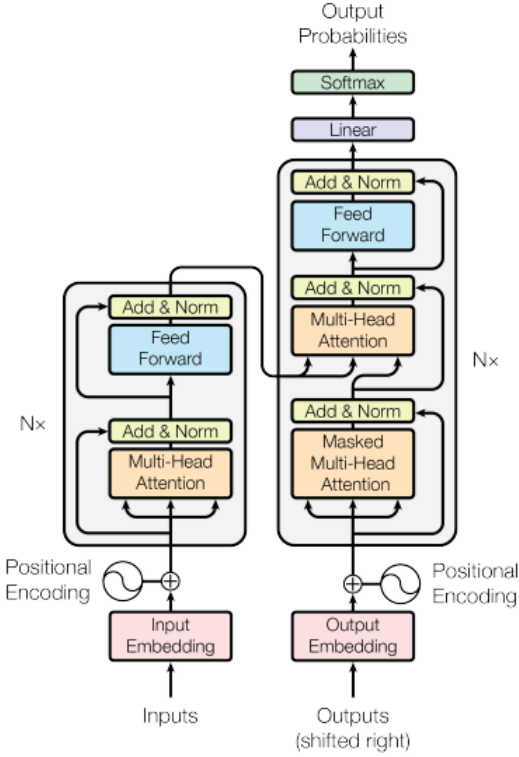


Fig. 1: Transformer Model Architecture

First, the inputs are embedded to vectors of size 512, then the positional encoding is added to the embeddings to inject information about the order of the sequences and relative positions, and the result is fed into the transformer.

The transformer uses two different types of attention: Scaled Dot-Product Attention and Multi-head Attention. The data is first fed to the scaled dot-product, which computes the attention function on a set of queries simultaneously, packed together into a matrix Q . The keys and values are also packed together into matrices K and V , and then, the Attention value is computed as follows:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

Multi-head attention, allows the model to jointly attend to information from different representation subspaces at different positions. Here, the keys, queries, and values are first linearly projected onto dimensions $h=8$, then each of h layers is computed in parallel and their outputs are concatenated and linear projected. This linear projection reduces the size of each

word vector, leading the architecture to learn more complex dependencies between words. The operation is shown below:

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O$$

$$where : head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

The decoder is auto-regressive at each step, meaning that it consumes the previously generated symbols as additional input when generating the next. It begins with a start token, and it takes in a list of previous outputs as inputs and the encoder outputs. The linear and softmax blocks compute the probability for the next word and return the highest probability as the next word. Finally, the decoder stops when it generates an output token.

The reason they used self-attention was for considering the total computational complexity per layer, the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required, and the path length between long-range dependencies in the network.

They trained their model on the standard WMT 2014 English-German dataset and WMT 2014 English-French dataset for the machine translation task. They used Relu activation, Adam optimizer, and Residual Dropout. Each layer employs a residual connection for backpropagation and prevents exploding gradients. This model achieved a new state-of-the-art for the two datasets and a great increase in performance and speed of computation.

In their paper, Transformer has only been applied to transduction model tasks. However, now transformers are widely used in many other tasks.

B. Vision Transformer

Inspired by the Transformer scaling successes in NLP, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale" [3], introduces Vision Transformer, a new state-of-the-art model for image classification. Several works have used transformers in the context of image processing in the past, and many of these specialized attention architectures demonstrate promising results on computer vision tasks. However, they require complex engineering to be implemented efficiently on hardware accelerators. The proposed model is trained in a supervised fashion on image classification. Images are transformed into sequences of fixed-size image patches representing "tokens", similar to word tokens in NLP, with an extra learnable "classification token" in the sequence. Then, each of them are linearly embedded, added by position embeddings, and finally, the resulting sequence of vectors are fed to a standard Transformer encoder. Fig. 2 shows details of the architecture.

Moreover, instead of using raw patches as the input sequence, the paper presented a hybrid model where the patch embedding projection is applied to patches extracted from a CNN feature map, but the rest of the architecture remains unchanged.

The ViT is first pre-trained on large datasets and fine-tuned to smaller datasets, then evaluated on the test set. For

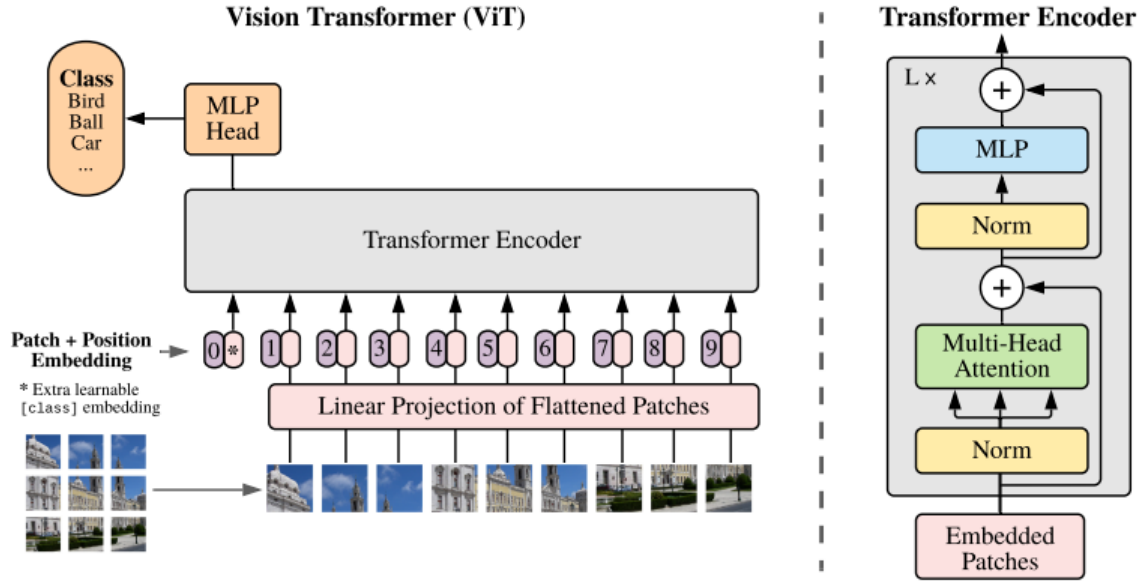


Fig. 2: Vision Transformer Model Architecture

the training part, they used three different datasets, including ImageNet (1K classes / 1.3M images), ImageNet-21K (21K classes / 14M images), JFT (18K classes / 303M images). For performance evaluation, they used a number of benchmark tasks, including ImageNet, CIFAR-10/100, the 19-task VTAB and etc. They also used different variants of ViT models: ViT-Base, ViT-Large, and ViT-Huge, which are different in the number of layers, hidden size D , MLP size, heads, and params. They compared the models with popular image classification benchmarks, such as Big Transfer and Noisy Student.

Results show that ViT models pre-trained on the JFT-300M dataset outperform ResNet-based baselines on all datasets while taking substantially less computational resources to pre-train. ViT pre-trained on the smaller ImageNet-21k dataset performs well too. However, ViT pre-trained on ImageNet-1k is a little worse than ResNet. This means without a big dataset for pre-training, ViT does not perform well. Table. I shows comparison with state of the art on popular image classification benchmarks (All models were trained on TPUv3 hardware).

TABLE I: Vision Transformers Results

	JFT (ViT-H/14)	JFT (ViT-L/16)	I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 $\pm$ 0.04	87.76 $\pm$ 0.03	85.30 $\pm$ 0.02	87.54 $\pm$ 0.02	88.4/88.5*
ImageNet Real.	90.72 $\pm$ 0.05	90.54 $\pm$ 0.03	88.62 $\pm$ 0.05	90.54	90.55
CIFAR-10	99.50 $\pm$ 0.06	99.42 $\pm$ 0.03	99.15 $\pm$ 0.03	99.37 $\pm$ 0.06	—
CIFAR-100	94.55 $\pm$ 0.04	93.90 $\pm$ 0.05	93.25 $\pm$ 0.05	93.51 $\pm$ 0.08	—
Oxford-IIIT Pets	97.56 $\pm$ 0.03	97.32 $\pm$ 0.11	94.67 $\pm$ 0.15	96.62 $\pm$ 0.23	—
Oxford Flowers-102	99.68 $\pm$ 0.02	99.74 $\pm$ 0.00	99.61 $\pm$ 0.02	99.63 $\pm$ 0.03	—
VTAB (19 tasks)	77.63 $\pm$ 0.23	76.28 $\pm$ 0.46	72.72 $\pm$ 0.21	76.29 $\pm$ 1.70	—
TPUV3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

Finally, the author conducted a preliminary study on masked patch prediction for self-supervised training ViT, and showed that through self-supervised pre-training, the accuracy increased by 2% in comparison to training from scratch. This paper showed that the computer vision task’s heavy reliance on CNNs is not necessary and a well pre-trained ViT can have a promising performance.

C. Sharpness Aware Minimization Optimizer

“Sharpness-Aware Minimization (SAM) for Efficiently Improving Generalization” paper introduces a novel, effective procedure for instead simultaneously minimizing loss value and loss sharpness. [4] Most modern neural networks are heavily over-parameterized, hence, optimizing the network weights to minimize a loss function into absolute minima may lead to over-fitting and losing generalization capabilities. For solving this problem, this paper’s approach is different from finding a weight vector, which minimizes the loss function for the given training set, using Gradient-Descent to determine the function global minima. Their reformulated optimization problem is locating the loss function minima surrounding. For this purpose, their algorithm is calibrating the network, also around that point, so that the loss function will output minimal values. This approach considers the geometric properties of the loss surface around the minima points. Their loss function contains two items: 1. the maximal loss in a small surrounding ϵ of the point w , and 2. a standard regularization item with L2 Norm. More details about solving the SAM problem can be found in the paper.

The experiments results proved that SAM improves model generalization across a variety of benchmark datasets (e.g., CIFAR-10, 100, ImageNet, finetuning tasks) and models, yielding novel state-of-the-art performance. Furthermore, SAM showed a significant improvement during training on datasets with noisy labels, since it is preventing convergence in sharp minima.

III. MODEL ARCHITECTURE

The model architecture is composed of two main parts.

- 1) 2D slice embedder
- 2) slices sequence model

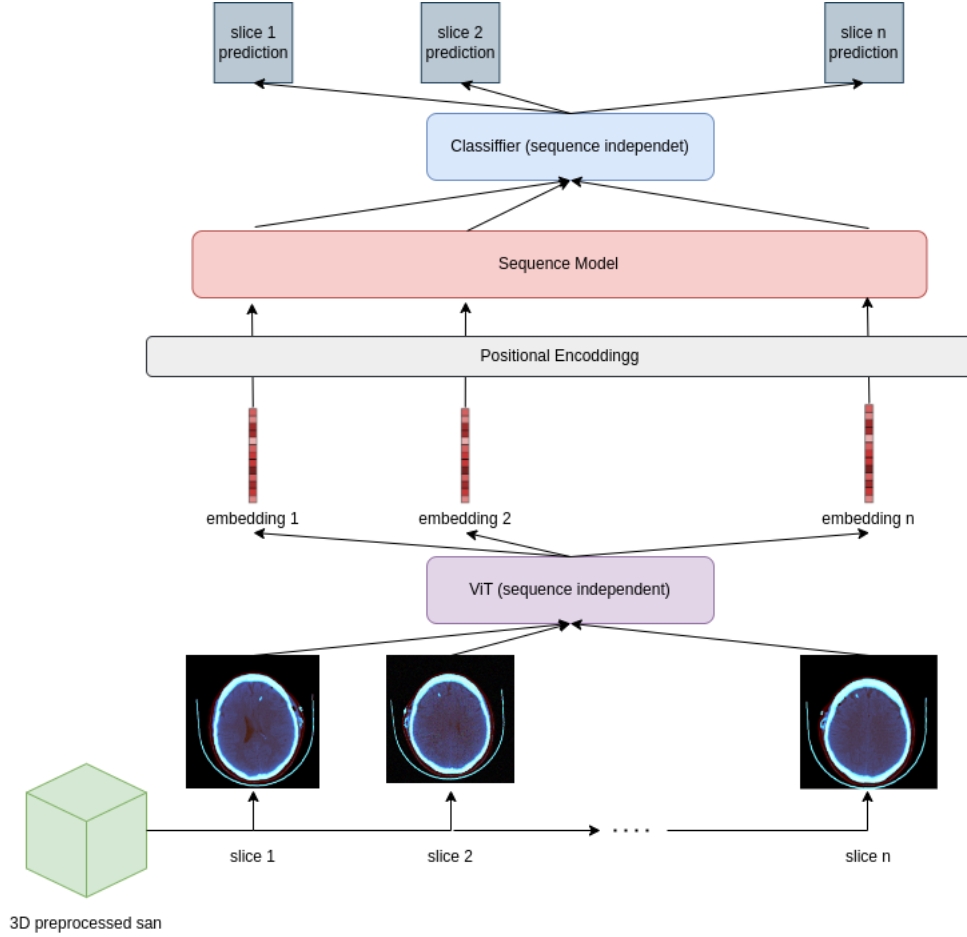


Fig. 3: 2.5D Model Architecture

The first one extracts features from 2D slices into a 128d embedding vector, and the second one extracts temporal features between these embeddings. All slices of a CT scan are passed through a same features extractor, and then the embeddings are fed into the sequence model. Fig. 3 shows the model architecture.

IV. DATA PREPARATION

Fig. 4 shows the preparation steps we applied to our data. The details are explained in the following sections.

A. Dataset

The dataset is provided by the Radiological Society of North America (RSNA®) and is composed of over 750,000 CT slices in the format of DICOM files, each one annotated for the presence and different types of ICH [5]. Apart from the image values, these DICOM files include some metadata that gives some specific information. Based on this metadata, we group 2D slices that share the triple (study ID, scan ID, patient ID) to make 3D scans, and in each scan, we sort the slices with respect to the image position. The result is over 25,000 CT scans. However, the number of slices in different scans vary in the range of 20-60 because of different imaging protocols.

Due to the lack of time for training the large dataset, we use randomly selected sub-samples of this dataset to train our model. To do so, we randomly pick 1000 scans of patients and 200 scans of healthy controls, and we split them into train, validation and test sets with 0.75, 0.15, 0.10 proportions respectively such that the ratio between patients and healthy controls are the same among the sets. Therefore, we are approximately using 4% of the available dataset, and having comparable results is considerable. Table. II shows details of this random subdataset.

TABLE II: subdataset details

#	Train	Validation	Test
patient scans	750	150	100
healthy control scans	150	30	20
with hemorrhage slices	9300	1650	1366
without hemorrhage slices	21308	4497	2711

B. Preprocessing

As mentioned above, our dataset is in the DICOM format, 16-bit images (with values ranging from -32768..32767), meaning that each voxel can store a large amount of information. Implementing proper windowing to the scans is

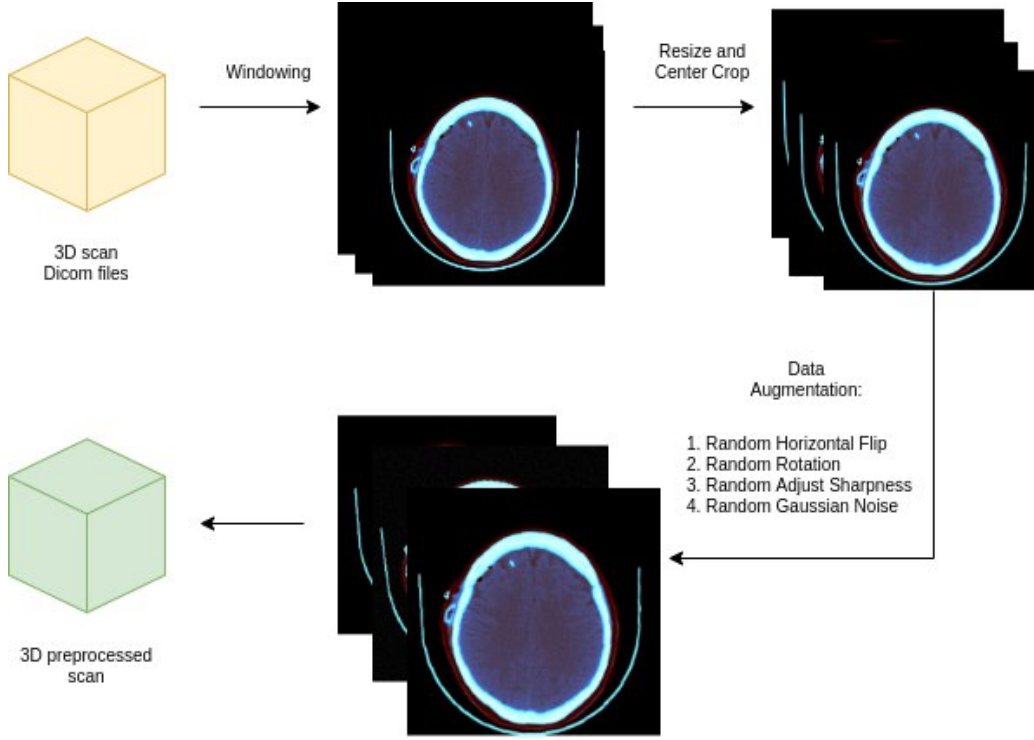


Fig. 4: Data Preparation Steps

an important factor in the workflow [6]. A window is an instruction to the computer to highlight only voxels which fulfill a specific value. Table. III shows details of our windows.

TABLE III: Windows details

Window	Width	Level
Brain Matter	80	40
Subdural	80	200
Bone	2800	600

The brain Matter window is able to pick up most abnormalities, however, other windows are also required for diagnosis, for example, the Bone window is useful to detect any bone fractures, which increases the probability of any hemorrhage in the brain. Fig. 5 shows an example of our windows.

After Windowing, we apply a resize to (400, 400) and a center crop of (348, 348) on all the dataset, since in our CT scans the edges are mostly black.

C. Data Augmentation

In the training loop, for preventing overfitting and increasing the diversity of data, we apply data augmentation using random horizontal flip, random rotation in the range of $[-5, +5]$, random adjust sharpness with factor of 1.2, and a random gaussian noise addition with mean=0 and std=0.02.

We don't apply the augmentation transforms directly on the train dataset. In that case, the size of our training dataset does not change, and we only manipulate some of them. Instead, we apply the augmentation inside the training loop. As a result,

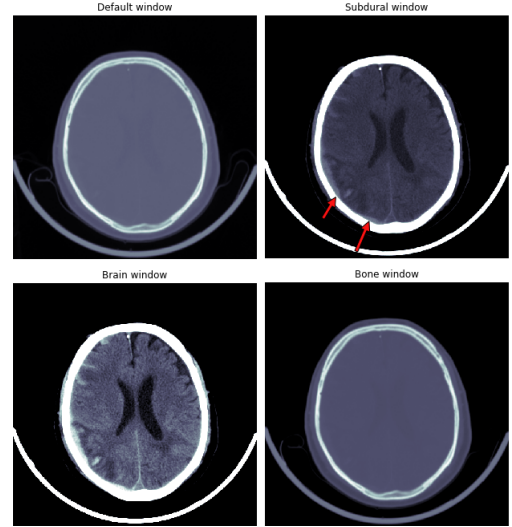


Fig. 5: Windows

in each epoch, it is highly possible that we have different set of training data.

V. TRAINING PARAMETERS

In the following, we define some parameters that we have used to train our model.

A. Loss Function

Loss function is a metric that evaluates how well the model performs in predicting the output correctly; If the prediction

deviates from the actual result, this function will return a large value and vice versa. Therefore, our goal is to modify the network in a way that results in a lower loss function. To achieve this, gradient-based optimizers are used.

In this paper, our task is to classify whether a slice contains hemorrhage or not, and thus we have two classes to classify. We compare training with two different loss functions: cross-entropy (CE) and binary cross-entropy (BCE). Additionally, because the dataset is unbalanced, a weighting scheme proportional to the inverse counts of each class is used in loss functions to give more satisfaction to rare class correct predictions.

1) *Cross Entropy*: cross-entropy function is used in multiclass classification setups. In this project, because we have two classes to classify, the model's final layer must have two cells whose values represent how likely the input belongs to the corresponding class. To have the probability of each class, the output is passed through a softmax layer, and then the probabilities are used in a cross-entropy function to compute the loss value. The formula is as follows:

$$CE(y, Y) = - \sum_{y \in O} Y \log(y)$$

where O is the set of output values, y is the probability of model having that output, and Y is the ground truth.

2) *Binary Cross Entropy*: It is a special case of cross entropy. When there are only two classes to classify. The difference is that we can represent the model output layer with a single cell. Its value is passed through a sigmoid layer to be mapped between 0 and 1. Based on how close the value is to 0 or 1, we can classify the input. The binary cross-entropy formula can be written as follows:

$$BCE(y, Y) = -(Y \log(y) + (1 - Y) \log(1 - y))$$

where y is the sigmoid output, and Y is the ground truth.

B. Optimizer

We train our model with two different optimizers: Adam and SAM. Due to the fact that transformers' loss functions are highly non-convex, we expect SAM optimizer to give us smoother train-validation curves, and better final test accuracy compared to the Adam optimizer. Also, to better avoid overfitting and increase the model generalizability, weight decay and momentum techniques are also used.

C. Learning Rate Scheduler

Learning rate controls the step-size that optimizer takes in the negative gradient direction of the loss function. It is very important to pick an efficient value for the learning rate because if it is picked larger, in case that loss function is highly non-convex, it will oscillate around the local minimum, and oppositely, if it is picked very smaller, it results in a very slow training. In this project, we initialize our learning rate by $1e-4$, which is not very small, and then we define a Reduce on Plateau (RoP) scheduler on it. At the end of each epoch, this scheduler divides the learning rate by 10 if training loss has not been decreased.

D. Batch size

Although mini-batch size may seem less important, based on the dataset, it can affect the result to overfit or not be trained well if it is not picked correctly. We inspect train-validation curves and also final test accuracy by using different batch sizes of 32, 64, and 128.

E. Early Stopping

The intuition behind having validation set is to control training parameters while training. In this project, we use Early Stopping with patience=3 in the first state, and patience=3 in the fine-tuning state meaning that it stops the training if the model overfits on train data, and validation loss does not decrease in some subsequent iterations. Also, we save the model with the best validation loss so far, and we use that model to compute the final test accuracy.

VI. EXPERIMENT

Training such a big 2.5D model needs a large dataset. However, we are using a small subdataset, and thus we do the training in two stages.

A. Training ViT

In the first step, we fine-tune a ViT to represent a good embedding of 2D slices. We picked ViT-Base model from <https://github.com/lukemelas/PyTorch-Pretrained-ViT/> repository which is pre-trained on imagenet21k, and finetuned on imagenet1k datasets. This ViT model embeds 32×32 non-overlapping patches into a single value using a convolutional layer and treats those values as sequences. Then, these sequences are passed through 12 transformer layers, and finally, a fully connected layer projects the last transformer output into a 128d vector. Next, we use a classifier at the top of this model to do the classification. Figure. 6 shows details of this 2d model.

B. Training Sequence Model

After we finetuned the ViT to produce efficient embeddings out of 2D images, we train the 2.5D model. To do so, first, we transfer and freeze the trained ViT, and we train the sequence model. Next, all layers are unfrozen and the whole model is finetuned on 3D scans. The sequence model used in this training is a transformer, and the model architecture has been explained in section III. We tried two different setups:

- 1) only a transformer encoder, and a fully connected layer to decode it
- 2) the whole transformer including an encoder-decoder to train the model with the teacher forcing method. In this technique, we use a greedy approach to estimate the test set output.

VII. RESULT

First, we trained the 2D ViT model with 4 combinations of SAM-Adam optimizers, and CE-BCE loss functions with batch-size=32. Next, we trained the model with the best loss function and optimizer in different batch sizes of 64 and 128

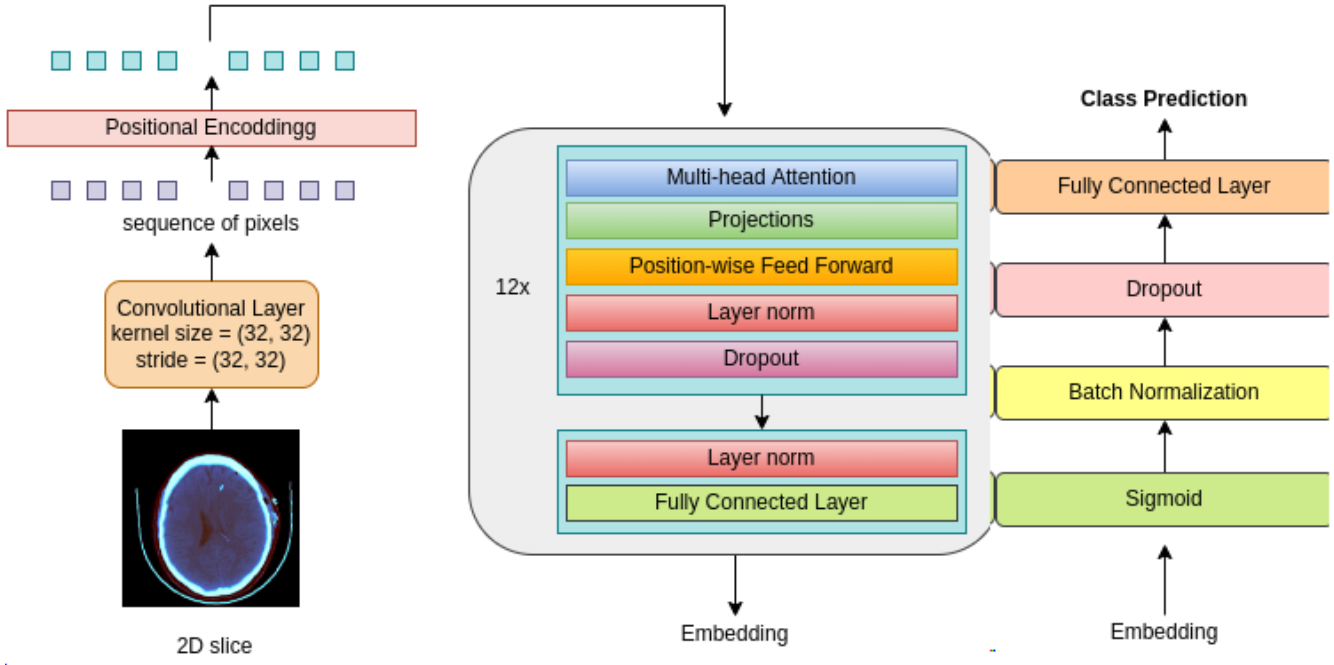


Fig. 6: 2D Model Architecture

to evaluate the effect of batch size. Fig. 7 shows the train-validation loss-accuracy curves with different setups. For each item in this figure, the curves in the first columns show the train-validation mean loss per epoch, and in the second column show train-validation mean accuracy per epoch. Also, the Final results can be seen in Table. IV.

Based on these results, the best training parameters are SAM optimizer, Binary Cross-Entropy Loss function, 32 Batch size, and using these parameters we have reached the accuracy of 85.06% to correctly detect slices with hemorrhage.

In the next step, we trained the 2.5D model using two sequence models.

- 1) transformer encoder
- 2) transformer encoder-decoder with teacher forcing

Figure. 8 shows the train-validation loss-accuracy curves, and Table. V shows the final result. As we can see, both these models resulted in drastic overfitting, and they could not be trained well. There are three reasons for that:

- 1) transformers are data-hungry, and stacking two transformers needs a much larger dataset to train while our subdataset is small.
- 2) the sequence length varies a lot between different samples as explained in section IV. Thus, the sequence model cannot be trained well, and 2D model seems to be more freindly to different imaging protocols.
- 3) there is an infinite number of different source embeddings (as they can get any 128d vector), but there are only three target embeddings, one for start-token, and two for our two classes.

TABLE IV: 2D ViT Model Result

Test Accuracy	Optimizer	Loss	Batch-size	
81.57	SAM	CE	32	
81.21	ADAM	CE	32	
85.06	SAM	BCE	32	
81.21	ADAM	BCE	32	
83.41	SAM	BCE	64	
82.68	SAM	BCE	128	

TABLE V: 3D ViT Transformer Model Result

Test Accuracy	Sequence Model
82.07	Transformer encoder
49.02	Transformer encoder-decoder with greedy aproach

VIII. CONCLUSION

In this project, we presented a double transformer model to process 3D CT scans, and to classify whether their slices are affected by hemorrhage or not. We did the training in two subsequent tasks. First, a ViT-Classifer model was finetuned to do the classification on 2D slices which resulted in 85.06% accuracy. Then, another transformer was deployed to find relationships between different slices of a CT scan. However, the latter approach result was not acceptable. The reasons behind that can be the size of the subdataset, variations in sequence lengths, and differences between source and target embeddings' dimensions. In the future, we are going to train different combinations of feature extractors and sequence models to see which one can give us a better result compared to 2D processing.

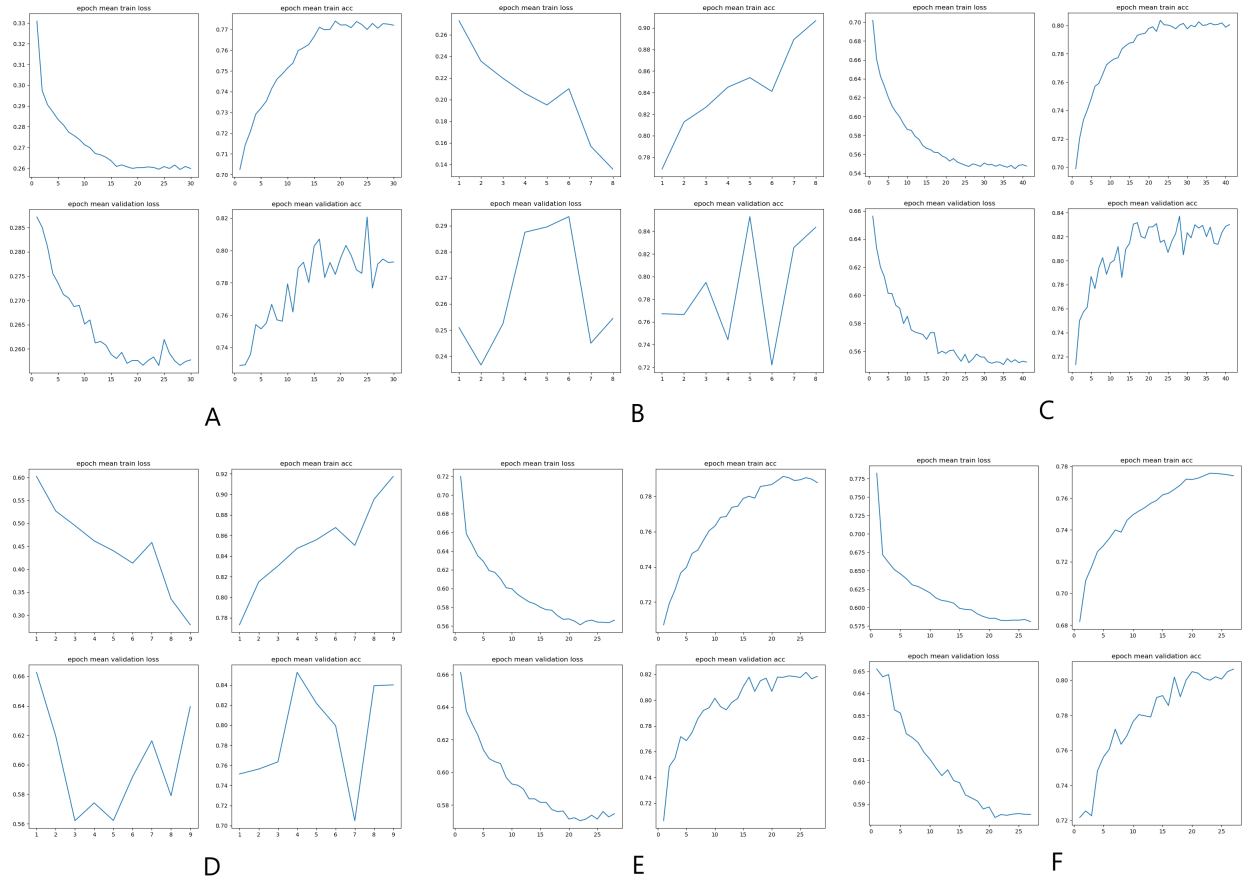


Fig. 7: 2D ViT Model Train and Validation Loss and Accuracy Curves.

A: optimizer=SAM, loss=CE, batch-size=32 B: optimizer=ADAM, loss=CE, batch-size=32 C: optimizer=SAM, loss=BCE, batch-size=32 D: optimizer=ADAM, loss=BCE, batch-size=32 E: optimizer=SAM, loss=BCE, batch-size=64 F: optimizer=SAM, loss=BCE, batch-size=128

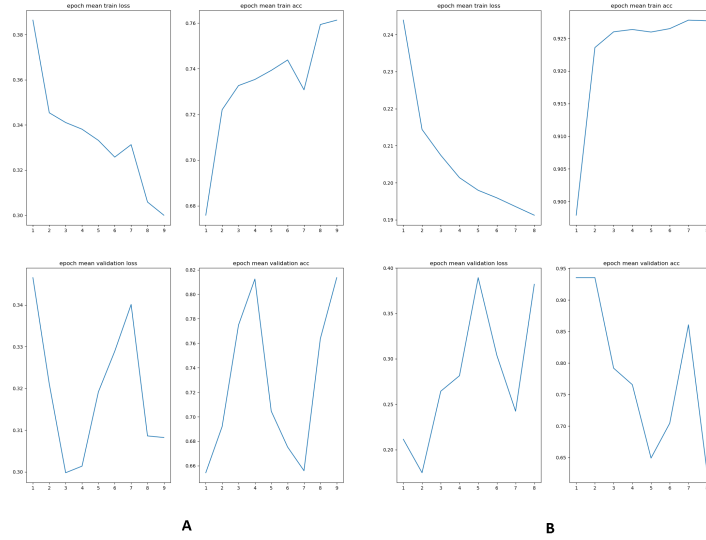


Fig. 8: 3D ViT Transformer Model Train and Validation Loss and Accuracy Curves.

A: Transformer encoder B: Encoder-decoder with teacher forcing

REFERENCES

- [1] Xue, Z.; Antani, S.; Long, L.R.; Demner-Fushman, D.; Thoma, G.R. Window classification of brain CT images in biomedical articles. In AMIA Annual Symposium Proceedings; American Medical Informatics Association: Bethesda, MD, USA, 2012; Volume 2012, p. 1023.
- [2] “Vaswani, A., Shazeer, N.M., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., & Polosukhin, I. (2017). Attention is All you Need. *Advances in neural information processing systems*, 30.”
- [3] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.
- [4] Foret, P., Kleiner, A., Mobahi, H., & Neyshabur, B. (2020). Sharpness-aware minimization for efficiently improving generalization. *arXiv preprint arXiv:2010.01412*.
- [5] “RSNA Intracranial Hemorrhage Detection.” <https://kaggle.com/c/rsna-intracranial-hemorrhage-detection>.
- [6] “Systematic Windowing.” <https://www.kaggle.com/code/dcstang/see-like-a-radiologist-with-systematic-windowing>.